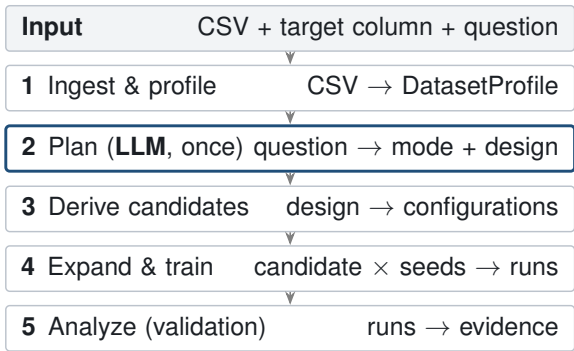


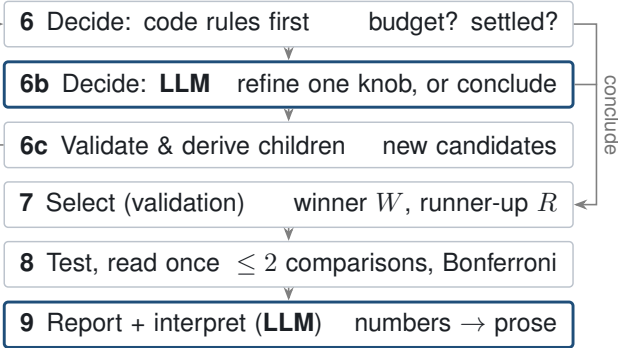
Adaptive ML Experiment Agent — what happens to a question

Every stage in the order the code runs it: what enters, the rules applied, what happens in the running example, and what the next stage receives.

SETUP – then every round



AFTER EACH ROUND – then the answer



The LLM is called at most four times per investigation: to turn the question into a design (2), to choose the next experiment after rounds 1 and 2 (6b, skipped whenever a code rule decides), and to explain the finished numbers (9). It never computes a statistic, never sees a data row, and never sees the test split before the comparisons are fixed. Every other stage is deterministic code.

RUNNING EXAMPLE — THE INPUT

Upload

diabetes_risk.csv, target column
diabetes_risk

Research question

“Which model performs best for classification on this dataset?”

Settings

MAX_ROUNDS = 3, N_SEEDS = 3

diabetes_risk.csv (15 000 rows, 19 columns)

patient_id	age	gender	bmi	...	hba1c_level	diabetes_risk
1	47	Female	26.5	...	7.3	Low
2	53	Female	20.5	...	6.9	Low

10 numeric features, 8 categorical; missing values in smoking_status (472), alcohol_consumption (3 788), income_bracket (463)

The stages below follow this *selection* investigation. The same code also runs *effect* investigations (“does X help?”); a real one is summarised after Stage 9.

1 Ingest & profile

deterministic, once per dataset

INPUT

The CSV text and the target column the user names (it is never guessed).

RULES

1. Rows with any missing value are dropped (no imputation); ≥ 20 rows and ≥ 2 target values must remain.
2. Task type: a non-numeric target, or ≤ 20 distinct integer-like values $\rightarrow$ **classification**; otherwise regression. A caller may override it.
3. Categorical features are one-hot encoded (a fixed vocabulary, so no leakage).
4. split_seed = 42: the same 70/15/15 train / validation / test rows for every run.
5. The cleaned CSV goes to data/uploads/<id>; the profile goes to PostgreSQL.

EXAMPLE

15 000 rows $\rightarrow$ **10 500** after dropping 4 500 rows with a missing value. Target text with 3 values $\rightarrow$ classification.

OUTPUT: DATASETPROFILE

```
task_type   : classification (inferred)
n_rows      : 10500
features    : 18 (10 numeric, 8 categorical)
n_features  : 49 (after one-hot)
classes     : Low 6302, Moderate 2630,
              High 1568
split       : 7350 / 1575 / 1575 rows
```

Majority-class rate (the sanity floor): $6302/10500 = 60.0\%$.

2 Plan

LLM once, then validated by code

INPUT: EXACTLY WHAT THE LLM RECEIVES

A fixed system prompt (the two modes, the four families and their knob ranges) and:

Research question: Which model performs best for classification on this dataset?

Dataset:

- task: classification, target column 'diabetes_risk'
- 10500 rows, 18 feature columns (10 numeric, 8 categorical)
- 3 classes, distribution {'Low': 6302, 'Moderate': 2630, 'High': 1568}

Counts only: no rows, no column names; at most 10 classes listed. 945 input tokens.

RULES

1. Strict JSON-schema output; one reply.
2. mode = selection: 2–4 distinct families.
mode = effect: one factor, 2–5 distinct levels, the reference among them, every value in range.
3. A broken rule is sent back once to fix; error set $\rightarrow$ PlanningError, the session fails with that sentence.

EXAMPLE: THE REPLY (SCRIPTED FOR THIS RUN)

```
{
  "error": null,
  "mode": "selection",
  "families": ["linear_baseline", "decision_tree",
              "random_forest", "mlp"],
  "factor": null, "levels": [], "reference": null,
  "base": {"model_type": "mlp", "normalize": null,
           "hyperparameters": {... all null ...}},
  "rationale": "Compare the four families at their defaults."
}
```

THE OTHER MODE: AN EFFECT DESIGN

```
{ "mode": "effect", "factor": "max_depth",
  "levels": ["2", "4", "8", "16"], "reference": "8",
  "base": {"model_type": "decision_tree", ...} }
```

Levels arrive as strings and are parsed per factor ("true" $\rightarrow$ boolean, "0.2" $\rightarrow$ 0.2).

3 Derive candidates

deterministic

INPUT

The design (2) and the family registry FAMILIES.

RULES

1. A **candidate** is one configuration minus the seed: family, hyperparameters, normalize, plus its lineage (parent, the one knob changed, its value, the round it was added).
2. **selection**: one root per family at its registry defaults and default preprocessing; the simplest chosen family (linear < tree < forest < MLP) is the reference c1.
3. **effect**: the reference level is the root c1; every other level is c1 with only the factor changed (parent c1).
4. The plan validates: unique ids, no two candidates with the same configuration, every value inside its range.

THE REGISTRY (BACKEND/MODELS/EXPERIMENT.PY)

family	knobs (range, default)	seeds	norm.
linear_baseline	none (logistic, max_iter 1000)	1	yes
decision_tree	max_depth 1–30 (8), min_samples_leaf 1–200 (1)	1	no
random_forest	200 trees; max_depth 2–40, min_samples_leaf 1–100 (1), max_features 0.1–1	3	no
mlp	1 hidden layer, ReLU; hidden_size 1–512 (64), dropout [0,1] (0), learning_rate (0,1] (0.001), batch_size 16–4096 (32), epochs 1–50 (20)	3	yes

OUTPUT: ROUND-1 CANDIDATES

```
c1 linear_baseline {} normalized, reference
c2 decision_tree  {max_depth 8,
                  min_samples_leaf 1}
c3 random_forest  {min_samples_leaf 1}
c4 mlp            {hidden_size 64, dropout 0,
                  learning_rate 0.001,
                  batch_size 32, epochs 20}
                  normalized
```

4 Expand & train

deterministic

RULES

1. configs_for: seeds 0, 1, 2 for a seeded family (random forest, MLP); one run for the seed-independent linear model and decision tree.
2. Every run loads the same split; StandardScaler is fit on the training rows only.
3. Every run scores **every** validation and test row, $i = 1..n$:

$$s_{r,i} = \begin{cases} \mathbf{1}[\hat{y}_{r,i} = y_i] & \text{classification} \\ (\hat{y}_{r,i} - y_i)^2 & \text{regression} \end{cases}$$
4. It also records the same metric on the training rows, the training loss and the time.
5. A run is failed only if it crashed or produced NaN/Inf; every other run is evidence. Each result is stored as it finishes.

EXAMPLE: ROUND 1

candidate	runs	time per run
c1 linear	1	0.2 s
c2 tree	1	0.2 s
c3 forest	3	1.6–3.2 s
c4 MLP	3	30–36 s (GPU, launch-bound)
		8

OUTPUT: ONE EXPERIMENT RESULT PER RUN

```
status      : ok
metrics     : accuracy, train_accuracy,
              train_loss, training_time_seconds
val_scores  : [1, 0, 1, ...] 1575 rows
test_scores : [0, 1, 1, ...] 1575 rows (sealed)
```

5 Analyze — validation only

deterministic, recomputed every time

RULES

For candidate c with successful runs R_c , average over seeds, then over rows:

$$\bar{s}_{c,i} = \frac{1}{|R_c|} \sum_{r \in R_c} s_{r,i}, \quad m_c = \frac{1}{n} \sum_{i=1}^n \bar{s}_{c,i}$$

Any two candidates a, b are paired by row:

$$d_i = \bar{s}_{a,i} - \bar{s}_{b,i}, \quad \hat{\Delta} = \frac{1}{n} \sum_i d_i$$

Resample the n rows with replacement $B = 2000$ times; the percentile interval is the CI:

$$CI_{1-\alpha} = [q_{\alpha/2}(\hat{\Delta}^*), q_{1-\alpha/2}(\hat{\Delta}^*)], \quad \alpha = 0.05$$

Verdict: *better* / *worse* if the CI lies entirely on one side of 0, else *inconclusive*. With $\sigma = +1$ (accuracy) or -1 (MSE):

$$L = \arg \max_c \sigma m_c$$

$$\mathcal{C} = \{c : \text{verdict}(c \text{ vs } L) \neq \text{worse}\}$$

- Pairs: every candidate vs the reference; in selection mode also vs the **leader** L and a refined child vs its **parent**.
- $\mathcal{C}$ = the **contenders**. Gap = σ (train metric – m_c); large means overfitting.
- Seed spread is reported as stability, never as the uncertainty.

EXAMPLE: AFTER ROUND 1 ($n = 1575$)

cand.	val	gap	vs leader c1, 95% CI
c1 linear	79.05 %	0.9	leader
c2 tree	75.8 %	7.6	–3.3 [–5.1, –1.6] worse
c3 forest	78.2 %	21.8	–0.85 [–1.90, +0.25] inconcl.
c4 MLP	77.4 %	7.6	–1.65 [–2.88, –0.38] worse

In percentage points. Seed spread: forest ± 0.5 , MLP ± 0.6 pts. The forest scores 100 % on its training rows.

OUTPUT: ANALYSIS

```
leader      : c1 (the reference itself)
contenders  : c1, c3 <- two families
conditions  : mean, seed_std, train, gap per id
comparisons : vs reference, vs leader
```

Uncorrected on purpose: these intervals choose the next experiment; they are never reported as a claim.

6 Decide

code rules, then LLM, then code

RULES, IN THIS ORDER

round = MAX_ROUNDS?

→ conclude, budget

all contenders one family?

→ conclude, settled

otherwise: ask the LLM

→ refine or conclude

A proposal is accepted only if (Plan.refine):

- selection:** the parent is a contender and the knob is a refinable knob of its family. **effect:** the knob is the factor, the parent the reference.
- 1–3 values, each in range.
- No configuration already run (seed-free key).

A violation goes back to the model once; still invalid → the session fails. Accepted values become children, trained next round.

EXAMPLE: WHAT THE LLM RECEIVES AFTER ROUND 1 (732 TOKENS)

```
{
  "question": "Which model performs best ...",
  "mode": "selection",
  "metric": "accuracy (higher is better)",
  "val_rows": 1575,
  "majority_rate": 0.6002,
  "reference": "c1",
  "leader": "c1",
  "round": 1,
  "rounds_left": 2,
  "candidates": [
    {
      "id": "c1",
      "family": "linear_baseline",
      "val": 0.7905,
      "train": 0.7999,
      "gap": 0.0094,
      "contender": true,
      "id": "c2",
      "family": "decision_tree",
      "val": 0.7575,
      "vs_leader": [-0.033, -0.0508, -0.0159],
      "contender": false,
      "id": "c3",
      "family": "random_forest",
      "val": 0.782,
      "seed_std": 0.0046,
      "train": 1.0,
      "gap": 0.218,
      "vs_ref": [-0.0085, -0.019, 0.0025],
      "vs_leader": [-0.0085, -0.019, 0.0025],
      "contender": true,
      "id": "c4",
      "family": "mlp",
      "val": 0.774,
      "contender": false
    ]
  ],
  "refinable": {
    "random_forest": {
      "max_depth": [2, 40],
      "min_samples_leaf": [1, 100],
      "max_features": [0.1, 1.0]
    }
  }
}
```

THE REPLY → DECISION

```
{
  "action": "refine",
  "parent": "c3",
  "knob": "min_samples_leaf",
  "values": ["5", "20"],
  "rationale": "The forest is a contender but its train-validation gap is large (overfitting) ..."
}
```

→ new candidates c5, c6 (parent c3), round 2

One compact row per candidate, no runs, no rows, no history. Proposing refine(c4, hidden_size, ...) here is rejected and sent back: "c4 is not a contender; refine one of ['c1', 'c3'] or conclude".

4–6 Rounds 2 and 3 — the lineage grows

the same stages again

EXAMPLE: CANDIDATE LINEAGE AT THE END

round 1 (roots)

round 2

round 3

c1 linear (ref)

c2 tree

c3 forest

c4 MLP

c5 leaf=5

c6 leaf=20

c7 depth=12

c8 depth=24

Every child differs from its parent in exactly one knob, so its “vs parent” difference is the effect of that knob.

EXAMPLE: VALIDATION EVIDENCE AFTER ROUND 3

candidate	val	gap	vs parent, 95% CI
c1 linear (leader)	79.05	0.9	—
c5 c3 + leaf 5	78.39	11.6	+0.19 [−0.32, +0.76]
c6 c3 + leaf 20	77.95	4.6	−0.25 [−0.95, +0.47]
c7 c5 + depth 12	78.48	9.5	+0.08 [−0.38, +0.53]
c8 c5 + depth 24	78.43	11.5	+0.04 [0.00, +0.11]

After round 2 the agent refined c5 (max_depth 12, 24). After round 3, round = MAX_ROUNDS: code concludes (budget), no LLM call. Contenders at the end: c1, c3, c5, c6, c7, c8; the tree and the MLP stayed clearly behind the leader, so they were never refined.

OUTPUT

```
8 candidates, 20 runs (8 + 6 + 6), 3 decisions:
  refine c3 min_samples_leaf (agent)
  refine c5 max_depth (agent)
  conclude (budget)
```

7 Select

deterministic, validation only

RULES

1. Winner W = the best *non-reference* candidate on validation.

2. Selection mode: runner-up R = the best candidate whose family is neither W 's nor the reference's. None $\rightarrow$ no second comparison.

3. Fixed now, before a single test score is read.

EXAMPLE $\rightarrow$ OUTPUT

$W = c7$ (forest, min_samples_leaf 5, max_depth 12; 78.5 %). Excluded families: forest, linear $\rightarrow R =$ best of tree / MLP = c4 MLP (77.4 %). The leader c1 is the reference itself, so W is its strongest alternative.

8 Test — the only read of the test split

deterministic

RULES

With k comparisons ($k = 1$ effect, $k \leq 2$ selection) each interval is computed at

$$1 - \frac{0.05}{k} = 95\% \text{ } (k = 1) \text{ or } 97.5\% \text{ } (k = 2)$$

(Bonferroni: the claims share one 5 % error budget.)

1. Primary: W vs the reference. Secondary: W vs R .

2. The same paired bootstrap as Stage 5, on the test rows.

3. Absolute scores m_W, m_{ref} with their own bootstrap CI.

4. Selection optimism: $\sigma(m_W^{val} - m_W^{test})$.

EXAMPLE: 1575 TEST ROWS, $k = 2$

W c7	79.6 % [77.6, 81.6]
reference c1	80.8 % [78.8, 82.8]

W vs reference	−1.2 pts [−2.3, −0.1] worse
W vs R (MLP)	+0.2 pts [−1.1, +1.6] inconclusive

val $\rightarrow$ test drop of W −1.1 pts (it scored higher on test)

Intervals at 97.5 %. The test split never influenced which candidates were trained, refined or compared.

9 Report

deterministic, then LLM for prose

RULES

1. The headline is built by code from the numbers (wording per verdict); when the reference led on validation, it says so first.

2. The report also stores both comparisons and confidence, absolute test scores, the drop, candidates per family (effort), rounds run, who stopped the loop (agent / budget / settled), the majority-class rate.

3. interpret (LLM, low reasoning effort) gets the finished report and writes 3–5 sentences that cite it. If it fails, the report is saved without it.

EXAMPLE: THE HEADLINE, AS GENERATED

linear_baseline was the best candidate on validation, so its strongest alternative was tested. On the held-out test set (1575 rows), random_forest, min_samples_leaf=5, max_depth=12 **performs worse than linear_baseline**: accuracy 79.6 % vs 80.8 %, difference −1.2 pts (97.5 % CI −2.3 pts to −0.1 pts). Against the strongest rival, random_forest, min_samples_leaf=5, max_depth=12 and mlp **are statistically tied**: difference +0.2 pts (97.5 % CI −1.1 pts to +1.6 pts).

OUTPUT: REPORT

effort	: linear 1, tree 1, forest 5, mlp 1
stopped_by	: budget rounds_run : 3
LLM calls	: 4 (945, 732, 960, 764 input tokens)
wall time	: 156 s (MLP runs dominate)

THE OTHER MODE, AND THE HARD LIMITS

AN EFFECT INVESTIGATION ON THE SAME DATA

“Does tree depth matter for a decision tree on this dataset?”
→ factor max_depth, levels 2, 4, 8, 16, reference 8; refine-
ments 6, 12, then 5, 7 (same factor, parent c1); concluded
by the budget.

level	val	vs depth 8 (95% CI)
2	73.9	−1.8 [−3.8, +0.2]
4	75.6	−0.2 [−2.0, +1.5]
5	76.7	+0.95 [−0.8, +2.5]
6	77.0	+1.27 [−0.1, +2.6]
7	76.8	+1.08 [−0.1, +2.3]
8 (ref.)	75.8	—
12	71.4	−4.3 [−6.0, −2.7] worse, gap 21 pts
16	71.6	−4.1 [−6.0, −2.2] worse, gap 27 pts

Test, $k = 1$: depth 6 vs 8, 78.9% vs 77.8%, +1.1 pts (95%
CI −0.3 to +2.4) → **no reliable difference**. 8 runs in 4 s
(the tree is seed-independent).

HARD LIMITS, ALL ENFORCED BY CODE

rounds	MAX_ROUNDS = 3
new candidates per decision	1–3
initial levels / families	2–5 / 2–4
runs per candidate	3 seeded, else 1
MLP cost per run	epochs ≤ 50 , batch ≥ 16
LLM calls	≤ 4 (plan, 2 decisions, interpret)
runs, selection	$\leq 8 + 2 \cdot 3 \cdot 3 = 26$
runs, effect	$\leq (5 + 2 \cdot 3) \cdot 3 = 33$
LLM output	$\leq 2\,048$ tokens, usage logged

Each LLM call is stateless: a fixed system prompt plus one
compact summary, so its size grows only with the (bounded)
number of candidates. Limitation: every result is conditional on
the one training split; capturing retraining variability would need
k-fold cross-validation, which is out of scope.